

# Contents

**Rapport Phase 1 — Étude du pattern “LLM Wiki” de Karpathy appliqué à Jarvis** **1**

△ Rappel cadre de mission . . . . . 1

Légende . . . . . 2

Section 1 — Comprendre le pattern LLM Wiki . . . . . 2

1.1 Origine et contexte . . . . . 2

1.2 Architecture en 3 couches . . . . . 2

1.3 Les 3 opérations canoniques . . . . . 3

1.4 Rôle de index.md et log.md . . . . . 4

1.5 Outils de support mentionnés par Karpathy . . . . . 4

1.6 Pourquoi pas RAG — la philosophie . . . . . 4

Section 2 — Pertinence pour Jarvis spécifiquement . . . . . 5

2.1 Persistance du contexte entre sessions . . . . . 5

2.2 Intégration avec le MCP Home Assistant existant . . . . . 5

2.3 Compatibilité Windows . . . . . 6

2.4 Coût en tokens avec un abonnement Claude Max . . . . . 6

2.5 Limite d'échelle ~100-200 pages — applicable à Jarvis ? . . . . . 6

Section 3 — Risques et points de vigilance . . . . . 7

3.1 Échecs documentés par la communauté . . . . . 7

3.2 Retour de production — Aaron Fulkerson (12/04/2026) . . . . . 7

3.3 Retour de production — Zafer Dace (17/04/2026) . . . . . 8

3.4 Ce qui pourrait mal tourner dans le contexte Jarvis . . . . . 8

3.5 Conditions pour que le pattern fonctionne . . . . . 8

Section 4 — Évaluation honnête . . . . . 9

4.1 Avantages réels vs complexité ajoutée . . . . . 9

4.2 Usage personnel/domotique vs recherche . . . . . 9

4.3 Alternatives plus simples à considérer . . . . . 9

4.4 Positionnement global . . . . . 10

Recommandation finale . . . . . 10

Synthèse décision . . . . . 10

Recommandation structurée . . . . . 10

Ce qu'il faut NE PAS faire . . . . . 11

Validation Mickael requise . . . . . 11

Sources consolidées . . . . . 11

# Rapport Phase 1 — Étude du pattern “LLM Wiki” de Karpathy appliqué à Jarvis

**Auteur** : Jarvis (assistant Home Assistant de Mickael) **Date** : 23 avril 2026 — Session 36 **Mission** : Étude préalable uniquement — **aucune modification d’architecture Jarvis Livrable** : Rapport d’analyse + recommandation Phase 2 go/no-go

## △ Rappel cadre de mission

Ce rapport est **informatif uniquement**. Aucune action sur l’architecture Jarvis (fichiers locaux, Home Assistant, skills, mémoire, MCP) n’a été effectuée pendant la production de ce document, et aucune ne le sera sans validation explicite de Mickael.

**Limitation technique à signaler** : le proxy réseau Cowork bloque l’accès direct (WebFetch) à gist.github.com, github.com, venturebeat.com, medium.com, pyshine.com, mindstudio.ai, antigravity.codes et substack.com. L’étude a donc été reconstruite via **WebSearch intensif** (12 recherches ciblées, ~40 résultats consolidés). Les citations directes de la gist originale de Karpathy n’ont pas pu être

récupérées verbatim, mais les reformulations convergentes croisées par 10+ sources secondaires (blogs techniques, implémentations open-source, retours de production) fournissent une couverture fiable du pattern. Les sources sont listées en fin de rapport.

Légende

- ✓ Confirmé / applicable
- △ Condition requise ou point d'attention
- ★ Limitation ou obstacle

Section 1 — Comprendre le pattern LLM Wiki

1.1 Origine et contexte

Andrej Karpathy (co-fondateur OpenAI, ex-Director of AI Tesla) a publié début avril 2026 un gist public intitulé “LLM Wiki” (identifiant 442a6bf555914893e9891c11519de94f). Ce document technique, appelé par Karpathy lui-même son “idea file”, décrit un pattern de base de connaissances personnelle maintenue par un agent LLM, pensé comme **alternative aux systèmes RAG classiques** (embeddings + base vectorielle + chunking).

✓ **Réception communauté très forte** : 5 000 stars GitHub en 4 jours, près de 3 000 forks, reprise par VentureBeat, Medium, DEV, Hacker News. Plus de 15 implémentations open-source apparues en moins de 3 semaines (claude-obsidian, llmwiki, obsidian-llm-wiki-local, second-brain, Karpathy-LLM-Wiki-Stack, karpathy-llm-wiki, llm-wiki-compiler, llm-wiki-mcp, obsidian-wiki, ekadetov/llm-wiki, Pratiyush/llm-wiki, etc.).

△ **Maturité réelle** : malgré cet engouement, le pattern est **émergent** — les premiers retours de production datent de quelques semaines seulement (Aaron Fulkerson 12 avril 2026, Zafer Dace 17 avril 2026). Aucune implémentation n’a encore atteint la stabilité d’un outil de prod pluriannuel.

1.2 Architecture en 3 couches

Le pattern repose sur une séparation stricte de 3 couches :

| Couche             | Rôle                                  | Qui écrit              | Qui lit                |
|--------------------|---------------------------------------|------------------------|------------------------|
| raw/               | Sources brutes immuables              | Utilisateur uniquement | LLM (lecture)          |
| wiki/              | Pages synthétisées et interconnectées | LLM uniquement         | Utilisateur + LLM      |
| CLAUDE.md (schema) | Règles, conventions, workflows        | Utilisateur (setup)    | LLM à chaque opération |

Détail d’une arborescence canonique (exemple issu de OmegaWiki repo et variants communautaires) :

```
llm-wiki/
├── CLAUDE.md          # Schema – le "cerveau" opérationnel du LLM
├── raw/               # Sources immuables (l'utilisateur ne les édite pas après dépôt)
│   ├── papers/       # PDFs, .tex, articles de recherche
│   ├── notes/        # Notes brutes .md prises par l'utilisateur
│   ├── web/          # Articles web clipés (Obsidian Web Clipper)
│   ├── discovered/   # Sources récupérées automatiquement (/daily-arxiv, /init)
│   └── tmp/          # Sidecars temporaires préparés par le LLM
```

|                 |                                                 |
|-----------------|-------------------------------------------------|
| └─ wiki/        | # Base maintenue par le LLM                     |
| └─ index.md     | # Catalogue navigable de tout le wiki           |
| └─ log.md       | # Journal chronologique des opérations          |
| └─ concepts/    | # Pages concept (une par idée transversale)     |
| └─ entities/    | # Personnes, projets, produits, organisations   |
| └─ papers/      | # Résumés structurés des papiers de raw/papers/ |
| └─ topics/      | # Cartes de domaines de recherche               |
| └─ summary/     | # Synthèses transversales multi-sources         |
| └─ foundations/ | # Connaissances pré-requises (pages terminales) |
| └─ ideas/       | # Idées de recherche avec leur cycle de vie     |
| └─ experiments/ | # Enregistrements d'expériences                 |
| └─ claims/      | # Affirmations testables                        |
| └─ outputs/     | # Artefacts générés (présentations, rapports)   |

✓ **Principe fondamental** : les fichiers de raw/ sont **immuables** — une fois déposés, on ne les édite plus. C’est la “source de vérité” qui garantit la traçabilité. Les pages de wiki/ sont au contraire **mutables** — le LLM les réécrit à chaque ingestion pour intégrer les nouvelles informations.

⚠ **Condition requisite** : l’architecture suppose que l’utilisateur accepte le principe du “**LLM is write-only over wiki/**” — si Mickael éditait manuellement une page wiki/, le prochain Lint pourrait écraser sa modification. C’est un contrat de délégation totale.

### 1.3 Les 3 opérations canoniques

**Ingest (l’opération la plus fréquente) Workflow** : 1. L’utilisateur dépose une source nouvelle dans raw/ (papier, article, note, URL clipée, transcript). 2. L’utilisateur demande au LLM (prompt explicite ou commande slash /ingest) de traiter cette source. 3. Le LLM **lit** la source, puis **discute** avec l’utilisateur les points-clés extraits (validation humaine optionnelle). 4. Le LLM écrit une **page de résumé** dédiée dans wiki/papers/ (ou équivalent). 5. Le LLM **met à jour l’index** (wiki/index.md) pour référencer la nouvelle page. 6. Le LLM **parcourt le wiki existant** et met à jour toutes les pages concept/entité liées (ajout de références croisées, mise à jour des contradictions, enrichissement). 7. Le LLM **ajoute une entrée** dans wiki/log.md (horodatage + résumé de ce qui a été ingéré + pages modifiées).

✓ **Ordre de grandeur** : une seule source touche typiquement **10 à 15 pages** du wiki (pattern documenté par Karpathy et confirmé par toutes les implémentations analysées). C’est ce qui crée l’effet “compounding” — chaque nouvelle source enrichit le réseau existant au lieu d’être isolée.

**Query Workflow** : 1. L’utilisateur pose une question au LLM (sans nécessairement pointer vers une source précise). 2. Le LLM **consulte** wiki/index.md d’abord, puis les pages pertinentes de wiki/. 3. Le LLM **synthétise** une réponse en citant les pages consultées. 4. **Optionnel mais recommandé** : le LLM **file la réponse** comme une nouvelle page du wiki (slug dérivé de la question). Futures sessions bénéficieront immédiatement de cette synthèse.

✓ **C’est le mécanisme de compounding** : une question à laquelle on a déjà répondu devient une ressource consultable. Karpathy l’appelle “**the compounding principle**”.

**Lint (opération de maintenance) Workflow** : 1. L’utilisateur déclenche périodiquement (**toutes les 2 à 4 semaines** d’après les recommandations) un audit complet. 2. Le LLM **lit tout le wiki** (ou autant qu’il peut tenir dans son contexte). 3. Le LLM **cherche** : - Les **contradictions** entre pages (la page A dit X, la page B dit non-X). - Les **affirmations périmées** (superseded claims — une source récente contredit une page ancienne). - Les **pages orphelines** (aucune page ne pointe vers elles). - Les **concepts manquants** (mention dans plusieurs pages mais pas de page dédiée). - Les **références croisées manquantes** (page A parle de X, page B parle de X, mais pas de lien). - Les **lacunes de données** (domaine sous-documenté alors que d’autres sont denses). 4. Le LLM **propose des corrections** (ou les applique automatiquement selon la politique configurée dans CLAUDE.md).

⚠ **Le Lint est le garde-fou principal du pattern** — sans lui, les erreurs d’ingestion s’accumulent silencieusement (voir Section 3).

1.4 Rôle de index.md et log.md

- ✓ **index.md** — **pièce angulaire de l’opération Query**. C’est un catalogue hiérarchique de toutes les pages du wiki, avec tags et brefs résumés. Le LLM le charge **à chaque début de session** pour savoir quoi explorer. Il est **toujours tenu à jour** par le LLM (touché à chaque Ingest).
- ✓ **log.md** — **journal append-only** qui trace l’histoire du wiki. Chaque ingestion, chaque lint, chaque query significative y laisse une trace datée. Sert à l’audit (savoir ce que le LLM a fait) et au debugging (retrouver quand une info est entrée dans le wiki).

⚠ **Enjeu technique majeur** : index.md doit **tenir dans le contexte du LLM** pour que Query fonctionne sans retrieval. C’est là que la limite d’échelle de 100-200 pages se matérialise (voir 1.6 et Section 3).

1.5 Outils de support mentionnés par Karpathy

| Outil                       | Rôle                                                                                                                      | Usage dans LLM Wiki                                                                                                                                                |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>qmd</b>                  | Moteur de recherche local pour fichiers markdown (BM25 hybride + vectoriel + re-ranking LLM, en local, CLI + serveur MCP) | Deviens utile <b>quand le wiki dépasse ~200 pages</b> et que index.md ne tient plus en contexte. Remplace alors la navigation directe par une couche de retrieval. |
| <b>Obsidian Web Clipper</b> | Extension navigateur (Chrome/Firefox/Safari/Edge/Brave) qui convertit une page web en markdown                            | Alimente raw/web/ en un clic de <b>Arctic</b> Brave (pertinent pour Mickael).                                                                                      |
| <b>Marp</b>                 | Générateur de présentations depuis markdown (thèmes, maths, export HTML/PDF/PowerPoint)                                   | Pour générer des <b>livrables</b> à partir du contenu wiki (ex. synthèse d’une recherche en slides).                                                               |
| <b>Dataview</b>             | Plugin Obsidian qui interroge le frontmatter YAML des pages                                                               | Permet au wiki d’afficher des <b>tableaux dynamiques</b> (ex. “toutes les pages tagées #TODO triées par date”).                                                    |

✓ **Obsidian lui-même** est recommandé comme **visualiseur** du wiki (graph view, backlinks natifs, écosystème de plugins). Le wiki reste du markdown plat — on peut le lire/éditer sans Obsidian (VSCode, Windsurf, Notepad).

1.6 Pourquoi pas RAG — la philosophie

Karpathy oppose deux philosophies :

**RAG classique** : - Upload de documents → chunking → embeddings → base vectorielle. - Sur chaque question : **re-découverte** depuis zéro. Retrieval → chunks pertinents → génération. - **Pas d’accumulation** : chaque requête est indépendante. - Synthèse de 5 documents = le LLM doit à chaque fois retrouver et assembler les fragments.

**LLM Wiki** : - **Compiler d’abord** : le LLM lit les sources une fois, synthétise en pages structurées interconnectées. - Sur chaque question : **requête contre l’artefact compilé**. - **Stateful** : la connaissance se construit par-dessus la précédente. - La synthèse se fait **une fois (+ incrémentale)**, les requêtes consomment une **encyclopédie pré-organisée**.

✓ **Argument fort** : Karpathy compare RAG à “exécuter du code source à chaque requête” alors que LLM Wiki serait “un binaire compilé”. L’analogie n’est pas parfaite mais elle capture l’essence.

⚠ **Limite de l’argument** : il suppose que la synthèse initiale est **correcte**. Si le LLM hallucine en ingérant, le mensonge est gravé dans le wiki pour toutes les requêtes futures (voir Section 3.1).

★ **Domaine de validité** : le pattern fonctionne pour des **corpus bornés et stables** (recherche personnelle, domaine d’expertise) — Karpathy le pose explicitement comme adapté à **~100 articles / 400K mots** maintenus par un utilisateur engagé. Il **n’est pas conçu** pour remplacer RAG dans un contexte entreprise avec millions de documents.

## Section 2 — Pertinence pour Jarvis spécifiquement

### 2.1 Persistance du contexte entre sessions

C’est le **besoin N°1 pour Jarvis** et l’axe sur lequel le pattern pourrait théoriquement apporter le plus.

**État actuel Jarvis** (constat sans jugement) : - memory/MEMORY.md = index manuel de la mémoire persistante. - memory/historique/AAAA-MM-JJ\_session\_NN\_titre.md = archives ponctuelles de sessions importantes (35 archives actuelles). - memory/historique\_reactif/AAAA-MM-JJ.md = logs d’alertes Mode Réactif. - .auto-memory/MEMORY.md = couche plus automatisée avec frontmatter typé (user/feedback/project/reference). - Ressources/Competences/\*.md = base de connaissances métier (HA, Gmail, Mode\_Réactif, Macros, etc.).

✓ **Point de convergence** : Jarvis a **déjà en place l’infrastructure de base** d’un LLM Wiki (arborescence markdown + index + logs datés + séparation archives/vivant). La refonte ne serait pas partir de zéro — elle serait une **discipline de maintenance plus systématique** appliquée à l’existant.

⚠ **Différences fondamentales à conscientiser** : - Le pattern Karpathy suppose que **le LLM écrit TOUT le wiki/** — or pour Jarvis, Mickael édite aussi manuellement beaucoup de fichiers (CLAUDE.md, TASKS.md, CONTEXTE.md, skills). Il faudrait clarifier ce qui est “wiki pur LLM-owned” vs “fichiers col-laboratifs”. - Le pattern prévoit raw/ **immuable** — Jarvis n’a pas cette séparation explicite. Les sources brutes (ex. une archive session) servent aussi parfois de référence active. - Le Lint hebdomadaire **n’existe pas** aujourd’hui dans Jarvis — les mises à jour de cohérence sont manuelles ou ad hoc en fin de session.

★ **Obstacle réel** : la mémoire de Jarvis est **cross-session mais pas auto-maintenue**. Le pattern suppose un LLM qui tourne en arrière-plan ou qui est appelé fréquemment. Jarvis fonctionne en **sessions synchrones** déclenchées par Mickael. Pour avoir un vrai effet “compounding”, il faudrait soit (a) transformer chaque session en mini-Ingest automatique, soit (b) planifier un Lint via Task Scheduler Windows ou scheduled task Cowork.

### 2.2 Intégration avec le MCP Home Assistant existant

**État actuel** : - MCP ha-mcp (add-on homeassistant-ai/ha-mcp, URL publique [https://mcp.might.ovh/private\\_Q49a0x](https://mcp.might.ovh/private_Q49a0x) expose 80+ outils ha\_\*. - C’est un MCP **métier Home Assistant** : entités, états, services, logs, automatisations, configuration.

**Ce que l’écosystème LLM Wiki apporte côté MCP** : - qmd existe en **serveur MCP** → pourrait être monté à côté de ha-mcp pour offrir une recherche hybride sur le wiki local. - Le thread Home Assistant Community #1005762 “[Filesystem MCP Server — expose your local directory to Claude \(Karpathy LLM wiki for Home Assistant\)](#)” documente explicitement un add-on HA dédié à ce cas d’usage. - Plusieurs MCP open-source existent spécifiquement pour LLM Wiki : flstevan87/llm-wiki-mcp (wiki\_read, wiki\_write\_page, wiki\_log\_append, wiki\_inventory), lucasastorian/llmwiki.

✓ **Compatibilité architecturale** : les deux mondes sont orthogonaux. ha-mcp gère le **runtime domotique**, un hypothétique wiki-mcp générerait la **connaissance persistante**. Aucun conflit technique.

⚠ **Doublon potentiel avec Cowork** : Mickael a **déjà** Read/Edit/Write/Bash natifs en Cowork Desktop sur l’arborescence. Un MCP Filesystem pour HA n’apporterait rien côté Cowork — son intérêt serait uniquement en **fallback Claude.ai** (mode chat sans accès fichiers) ou pour permettre à **Jarvis côté HA** (conversation agent HA natif, voice assistant) de lire le wiki.

★ **Friction structurelle** : Mickael a choisi S26 de découpler Cowork (cerveau) et Claude Code CLI (mains) car Cowork ne charge pas les MCP stdio. Ajouter un MCP wiki stdio reproduirait le même piège. Un MCP HTTP/SSE natif (comme ha-mcp) serait plus cohérent — l’add-on HA Community existe précisément pour ça.

2.3 Compatibilité Windows

✓ **Entièrement compatible** : - Obsidian : client natif Windows (download officiel). - Claude Code CLI : déjà installé sur le PC de Mickael (skill install-claude-code-windows, session 17). - qmd : CLI + MCP server, supposé multiplateforme (non vérifié, mais binaire Rust probable). - Dataview/Marp/Web Clipper : plugins Obsidian multiplateformes. - Le wiki lui-même = markdown plat → zéro dépendance OS.

⚠ **Un détail à valider** : l’add-on HA Filesystem MCP Server (thread #1005762) a été conçu pour exposer un dossier **local à HA** (donc sur le serveur HA). Si Mickael voulait utiliser son PC Windows comme host du wiki (accès Cowork + Obsidian natifs), il faudrait soit (a) monter le dossier Windows vers HA via SMB/WebDAV, soit (b) héberger le wiki sur l’Odroid HA et y accéder depuis Windows via partage réseau. Les deux sont faisables mais non triviaux.

2.4 Coût en tokens avec un abonnement Claude Max

✓ **Plan Max inclut Claude Code CLI headless** (auto-memory reference\_plan\_max\_includes\_cli) → les tokens d’Ingest/Query/Lint en CLI **comptent dans le quota Max** sans facturation API \$\$.

**Estimation de consommation** (ordres de grandeur issus des retours communautaires) :

| Opération                     | Fréquence                  | Tokens estimés par run                                         | Impact quota Max             |
|-------------------------------|----------------------------|----------------------------------------------------------------|------------------------------|
| Ingest d’une source moyenne   | ~1-5 par semaine           | 20k-60k tokens (lit source + 10-15 pages wiki + écrit updates) | Faible si quota Max respecté |
| Query enrichie + compounding  | Variable                   | 10k-30k tokens par question                                    | Négligeable en usage normal  |
| Lint complet (100 pages wiki) | 1× toutes les 2-4 semaines | 100k-200k tokens                                               | Significatif mais ponctuel   |
| Lint complet (200+ pages)     | 1× toutes les 2-4 semaines | Risque overflow 200k — besoin qmd                              | <b>Point critique</b>        |

⚠ **Zafer Dace** (retour terrain 17/04/2026) rapporte que son wiki atteignait **200-400K tokens totaux à 80-100 articles** — et que Claude commençait à donner des réponses **“confident but wrong”** en mélangeant des notes non reliées (“information blending”). Au-dessus de ~50-80 entrées, il a dû ajouter une couche RAG pour garder la précision.

★ **Limite technique claire** : Claude 4.6/4.7 a un contexte **200k tokens** (ou **1M pour le modèle 1M utilisé par Jarvis actuellement**). En 1M, la marge est **beaucoup plus grande** — mais (a) tous les modèles ne sont pas 1M, (b) la dégradation en milieu/fin de contexte (“context rot”) reste documentée indépendamment de la taille max.

2.5 Limite d’échelle ~100-200 pages — applicable à Jarvis ?

**Comptage approximatif de l’arborescence Jarvis actuelle** (estimation rapide sans scan exhaustif, à valider si décision Phase 2) :

| Catégorie                                                             | Fichiers .md estimés         |
|-----------------------------------------------------------------------|------------------------------|
| Racine (CLAUDE.md, TASKS.md, CONTEXTE.md, METRIQUES.md, README.md...) | ~5-8                         |
| .claude/skills/*/SKILL.md                                             | ~20-25                       |
| Ressources/Competences/                                               | ~15-20                       |
| Ressources/Protocoles/                                                | ~5-10                        |
| Ressources/Mode_Chat/ et Ressources/Cowork/                           | ~5                           |
| memory/historique/ (35 archives) + memory/historique_reactif/         | ~40-50                       |
| .auto-memory/*.md                                                     | ~40-45                       |
| Projets/, Runtime/, scripts, configs                                  | variable                     |
| <b>Total ordre de grandeur</b>                                        | <b>~130-165 fichiers .md</b> |

★ **Constat critique** : Jarvis est **déjà à la limite haute** de la zone de confort du pattern LLM Wiki (~100-200 pages). Si on implémentait le pattern tel quel, **on démarrerait très proche du seuil de rupture** — la marge de croissance serait faible avant de devoir ajouter qmd ou un équivalent.

⚠ **Nuance** : tous les .md de Jarvis ne sont pas équivalents à des pages wiki canoniques. Beaucoup sont des **logs** (archives historique) et non des pages de connaissance synthétisées. Dans une version “LLM Wiki-fiée” de Jarvis, on pourrait séparer : - raw/ = archives historiques immuables (~50 fichiers) - wiki/ = synthèses vivantes (CLAUDE.md, skills, compétences, auto-memories) (~80-100 fichiers)

Ce découpage rentrerait dans la zone de confort, mais demanderait un **travail de refonte non trivial** (classification de l’existant + renommages + réécriture d’index).

## Section 3 — Risques et points de vigilance

### 3.1 Échecs documentés par la communauté

**Trois modes de défaillance canoniques** (cités par Karpathy lui-même dans le thread du gist, puis repris par Aaron Fulkerson et d’autres) :

★ **Error accumulation and drift** — le LLM écrit une petite erreur en page A lors d’un Ingest → lors d’une prochaine Ingest touchant 10 pages liées, il propage l’erreur. Au bout de N ingestions, une incohérence silencieuse s’est installée. Le Lint aide **mais ne rattrape pas tout** : le Lint lui-même peut laisser passer des erreurs subtiles.

★ **Partial context pendant les updates** — le LLM ne peut pas relire **tout le wiki** à chaque Ingest (trop de tokens). Il travaille sur un sous-ensemble, ce qui signifie qu’une mise à jour peut rater une page concernée mais non chargée dans le contexte.

★ **Information loss par compression** — résumer = perdre. Ce qui est dans la page wiki n’est pas tout ce qu’il y avait dans la source brute. Pour un usage exploratoire c’est OK, pour un usage d’audit (retrouver le mot exact d’un document) c’est un problème.

### 3.2 Retour de production — Aaron Fulkerson (12/04/2026)

A. Fulkerson a opéré un LLM Wiki en prod pendant quelques semaines. Premier Lint après “des centaines de fichiers” : - **23 fichiers orphelins** (aucune page ne pointait vers eux). - **11 références croisées cassées** (pages mentionnées mais inexistantes). - **Impossible de savoir** quelles pages avaient “dérivé” de leur état initial correct sans un audit humain complet.

✓ **Leçon** : le Lint est **indispensable**, pas optionnel. Ne pas le planifier = garantir la dérive silencieuse.

### 3.3 Retour de production — Zafer Dace (17/04/2026)

Zafer Dace (DEV Community) a implémenté le workflow exact de Karpathy : - ★ **Échec à 80-100 articles** : total ~200-400K tokens, Claude commençait à “skimmer” et à fusionner des notes disparates (information blending). - ★ **Réponses “confident but wrong”** : le modèle hallucinait des connexions entre concepts non reliés. - △ **Solution adoptée** : RAG par-dessus le wiki → réduction de la charge contextuelle de 50 000 tokens à ~2 500 tokens par requête (facteur **20-40x**). Mais c’est **exactement ce que Karpathy voulait éviter**.

### 3.4 Ce qui pourrait mal tourner dans le contexte Jarvis

Projection des risques sur l’installation actuelle de Mickael :

★ **Écrasement de fichiers vivants critiques** — si l’opération Ingest est mal cadrée, le LLM pourrait réécrire CLAUDE.md, TASKS.md ou une skill active avec une synthèse erronée. **Règle 0 Jarvis appliquée** : confirmation avant écrasement obligatoire, mais si on automatise l’Ingest, la confirmation disparaît.

★ **Perte de la fidélité historique** — memory/historique/ est conçu comme archive **non-réécrite** (règle S33 feedback\_projets\_vs\_runtime). Un Lint agressif pourrait “nettoyer” ces archives jugées redondantes, **violant la fidélité aux sessions passées**.

★ **Explosion du coût tokens à chaque démarrage** — le CLAUDE.md actuel de Jarvis fait déjà **~550 lignes**. Si on l’enrichit avec une section “schema LLM Wiki” complète + workflows Ingest/Query/Lint, on risque de **consommer 30-50k tokens juste au chargement** de chaque session, avant même que Mickael ait posé une question.

★ **Conflit avec la Règle 0 (données sensibles)** — si l’Ingest automatise le parcours de raw/, rien n’empêche le LLM de traiter par inadvertance un fichier contenant des tokens/credentials. Il faudrait une **allowlist/denylist** stricte au niveau raw/.

△ **Duplication avec l’existant** — Jarvis a déjà : - Un système d’archives datées (memory/historique/). - Un système d’auto-memory typé (user/feedback/project/reference). - Des skills auto-déclenchées par description. - Un CLAUDE.md orchestrateur.

Superposer un LLM Wiki risquerait de **créer une 2ème source de vérité** qui dérive de la 1ère. À éviter absolument.

### 3.5 Conditions pour que le pattern fonctionne

D’après la consolidation des retours (Karpathy + Fulkerson + Dace + implémentations open-source) :

- ✓ **Condition 1** : Le corpus doit être **borné et stable** (~100 pages, croissance lente). Pas un flux continu d’infos volatiles.
- ✓ **Condition 2** : L’utilisateur doit **accepter la délégation d’écriture** au LLM sur wiki/. Pas d’édition manuelle concurrente.
- ✓ **Condition 3** : Le **Lint doit être planifié** (pas ad hoc). 2-4 semaines max entre deux passes.
- ✓ **Condition 4** : Le **schema (CLAUDE.md) doit être très précis** — conventions de nommage, templates de pages, workflows détaillés. Sinon le LLM improvise et la structure dérive.
- ✓ **Condition 5** : Au-delà de ~150 pages, **ajouter qmd** (ou équivalent BM25+vector) comme couche de retrieval pour préserver la précision.



## Section 4 — Évaluation honnête

### 4.1 Avantages réels vs complexité ajoutée

**Avantages potentiels pour Jarvis** : - ✓ Formalisation de la boucle **Ingest/Query/Lint** que Jarvis fait déjà en partie mais de manière informelle. - ✓ Introduction d’une **séparation raw/ immuable vs wiki/ vivant** — bénéfice de traçabilité. - ✓ Outils utiles (Obsidian Web Clipper pour sauver facilement des pages web en sources, Dataview pour dashboards de mémoire). - ✓ Meilleure visualisation de la connaissance via **Obsidian Graph View** (actuellement impossible sur l’arborescence Jarvis). - ✓ Discipline de **Lint hebdomadaire** → réduction de la dette de mémoire qui s’accumule entre sessions.

**Complexité ajoutée** : - ⚠ **Refonte de l’arborescence existante** (classifier ~130-165 .md en raw/wiki/autre) — travail de plusieurs heures. - ⚠ **Nouveau schema CLAUDE.md** — enrichir le v3.4 actuel avec les conventions Wiki (ou ajouter un CLAUDE\_WIKI.md séparé). - ⚠ **Installation + configuration d’Obsidian** sur le PC (vault pointant vers l’arborescence Jarvis). - ⚠ **Installation + configuration de qmd** (si > 150 pages prévues rapidement). - ⚠ **Création de commandes slash /ingest, /query, /lint** (actuellement inexistantes dans les skills Jarvis). - ⚠ **Setup scheduled task Lint** (2-4 semaines) via Task Scheduler Windows ou Cowork. - ⚠ **Risque de régression** pendant la migration (certaines skills/auto-mémories actuelles pourraient être cassées si mal redirigées).

**Bilan complexité / bénéfice** : la complexité d’implémentation est **réelle** (estimation grossière : 10-20h de travail réparties sur 3-5 sessions), et le bénéfice net est **incertain** car Jarvis fait déjà beaucoup de ce que le pattern propose — mais avec moins de discipline formelle.

### 4.2 Usage personnel/domotique vs recherche

Le pattern a été pensé par Karpathy pour un **chercheur IA consommant des papers** : - ~100 papiers, 400K mots, vocabulaire technique stable. - Questions du type “quelles sont les 3 architectures principales testées dans mon corpus sur X ?”. - Connaissance **encyclopédique**, pas **opérationnelle**.

L’usage Jarvis est **différent** : - **Opérationnel** : “débloquer IP HA bannie”, “planifier tri email”, “proposer un script pour une automation”. - **Configuration** : état des entités HA, secrets, credentials, historique d’incidents. - **Mémoire collaborative** : ce que Mickael a décidé, ce qu’il préfère, ce qu’il faut éviter. - Cycle court (session courante) + long (préférences durables) + archive (traçabilité).

★ **Mismatch partiel** : le pattern LLM Wiki est excellent pour **l’encyclopédie personnelle de concepts**, moins pour la **mémoire opérationnelle d’un assistant technique**. Les skills Jarvis actuelles (pédagogie, workflows, commandes précises) ne sont pas des “pages encyclopédiques” au sens Karpathy.

⚠ **Zone où le pattern serait pertinent pour Jarvis** : l’enrichissement de Ressources/Compétences/ (Home\_Assistant.md, Gmail.md, Mode\_Reactif.md...) pourrait effectivement bénéficier d’une discipline Wiki — chaque nouvelle technique apprise serait ingérée, chaque contradiction détectée par Lint. C’est un **sous-ensemble** du périmètre Jarvis, pas le tout.

### 4.3 Alternatives plus simples à considérer

**Alternative A — Améliorer l’existant sans refonte** : - Conserver l’arborescence actuelle. - Ajouter une **skill jarvis-lint** qui fait un audit périodique de cohérence sur Ressources/ + .claude/skills/ + .auto-memory/ (pas sur memory/historique/). - Planifier cette skill via scheduled task (toutes les 2-4 semaines). - Coût implémentation : ~2-4h. Bénéfice : **la discipline Lint sans changer l’architecture**.

**Alternative B — Pattern sélectif sur un sous-dossier** : - Créer un **nouveau dossier dédié wiki/** (vide au départ) structuré selon Karpathy. - L’utiliser pour **un usage précis** — par exemple : “tout ce que Jarvis apprend sur la domotique au-delà du setup existant de Mickael”. - Garde le reste de Jarvis intact. - Coût : modéré. Bénéfice : **tester le pattern sur un périmètre limité avant généralisation**.

**Alternative C — Obsidian en visualiseur pur :** - Installer Obsidian et pointer un vault vers l'arborescence Jarvis existante. - Profiter de **Graph View**, backlinks et Dataview **sans changer l'architecture**. - Ajouter progressivement des [[wikilinks]] dans les fichiers pour enrichir le graphe. - Coût : minimal (~30 min). Bénéfice : **gain de visualisation immédiat sans risque**.

**Alternative D — Ne rien changer :** - Considérer que Jarvis fait **déjà** l'essentiel de ce que le pattern apporterait. - Documenter simplement dans une auto-memory “pattern LLM Wiki étudié, non retenu, raisons X/Y/Z”. - Coût : 0. Bénéfice : **stabilité, pas de régression**.

#### 4.4 Positionnement global

✓ Le pattern Karpathy est **intellectuellement solide** et **confirmé en production à petite échelle**.

△ Il est **émergent** (< 1 mois de retours terrain), avec des failure modes déjà bien identifiés mais pas encore complètement mitigés par l'outillage communautaire.

★ Sa **limite d'échelle** (~100-200 pages) **rogne la marge de croissance** pour Jarvis qui est déjà dans cette zone.

★ Sa **philosophie de délégation totale d'écriture** au LLM est **en tension** avec le mode de travail collaboratif actuel Mickael ↔ Jarvis (où Mickael édite aussi manuellement beaucoup de fichiers clés).

△ Les **outils d'écosystème** (qmd, claude-obsidian, Obsidian Web Clipper, Marp, Dataview) sont **individuellement pertinents**, indépendamment du pattern complet — on pourrait en adopter certains sans adopter tout le pattern.

### Recommandation finale

#### Synthèse décision

**Le pattern LLM Wiki de Karpathy n'apporte pas un gain suffisant pour justifier une refonte complète de l'architecture Jarvis en Phase 2.** Il apporte **quelques idées exploitables ponctuellement**, mais la migration globale aurait un rapport bénéfice/risque défavorable.

#### Recommandation structurée

**× Phase 2 “Refonte complète LLM Wiki de Jarvis” : NON.**

Raisons principales : 1. ★ Jarvis est déjà à la limite haute de la zone de confort du pattern (~130-165 .md, zone 100-200 où les failure modes commencent). 2. ★ Jarvis fait déjà l'essentiel (index persistant, archives datées, auto-memory typée, skills auto-déclenchées) — la refonte créerait une **2ème source de vérité** qui dériverait de la 1ère. 3. ★ Mismatch de philosophie : Jarvis est un assistant **opérationnel collaboratif**, le pattern est conçu pour une **encyclopédie personnelle déléguée**. 4. △ Risques réels de régression pendant la migration (~10-20h de travail, skills et auto-memories à redirectionner). 5. △ Le retour de prod Zafer Dace (panne à 80-100 articles) documente un risque concret si on dépasse le seuil en cours de vie.

✓ **Phase 2 alternative “Adoption sélective” : OUI, si Mickael souhaite avancer, dans l'ordre suivant.**

**Priorité 1 — Obsidian en visualiseur pur (1 session, ~30-45 min) :** - Installer Obsidian sur le PC Windows. - Créer un vault pointant vers D:\Might\IA\Projets Cowork\Jarvis - Home Assistant\ - Explorer Graph View pour visualiser les liens déjà implicites entre fichiers. - Décision après test : garder/abandonner. **Zéro changement d'architecture.**

**Priorité 2 — Skill jarvis-lint (2 sessions, ~3-5h) :** - Créer .claude/skills/jarvis-lint/SKILL.md implémentant un audit périodique sur Ressources/ + .claude/skills/ + .auto-memory/ (PAS sur memory/historique/ qui reste intouchable — règle S33). - Planifier via scheduled task ou Task Scheduler

(2-4 semaines). - Modèle : inspiré du Lint Karpathy mais adapté à la structure Jarvis existante. - Bénéfice : **la discipline Lint sans risque de refonte**.

**Priorité 3 (optionnelle, à décider après Priorité 2) — Obsidian Web Clipper + dossier Wiki/sélectif (1 session, ~2h)** : - Installer Obsidian Web Clipper dans Brave. - Créer un dossier Wiki/dédié (vide au départ) pour tester le pattern sur un périmètre limité (ex. “tout ce que j’apprends de nouveau sur la domotique avancée qui ne rentre pas encore dans les Compétences existantes”). - Laisser les autres dossiers Jarvis intacts. - Bénéfice : **tester le pattern sur un sandbox**, décider après 1-2 mois d’usage si on élargit.

### Ce qu’il faut NE PAS faire

- ★ **Ne pas transformer** Ressources/ en wiki/ Karpathy — ça casserait les skills auto-déclenchées.
- ★ **Ne pas faire réécrire** memory/historique/ par le LLM — règle de fidélité aux archives (S33).
- ★ **Ne pas adopter qmd tout de suite** — utile uniquement si le wiki dépasse 150-200 pages réelles, pas encore le cas.
- ★ **Ne pas installer claude-obsidian** (le repo AgriciDaniel) clé en main — il suppose un vault vierge avec sa propre structure, incompatible avec l’arborescence Jarvis existante.

### Validation Mickael requise

**Si Mickael valide la recommandation “Phase 2 Adoption sélective”** : - Prochaine étape = session dédiée à **Priorité 1 uniquement** (test Obsidian visualiseur). - Bilan après 1 semaine d’usage Obsidian → décision d’aller ou pas vers Priorité 2. - **Aucune modification de l’architecture Jarvis lors de Priorité 1** (juste l’ajout d’un client Obsidian qui lit les mêmes fichiers).

**Si Mickael préfère “Ne rien changer”** : - Je documente cette étude dans une auto-memory (project\_llm\_wiki\_etudie\_non\_retenu.md) pour traçabilité. - Je classe ce rapport dans memory/historique/ comme archive de décision. - Zéro modification.

**Si Mickael veut creuser un point précis** (ex. “en fait seul le Lint m’intéresse”) : - Je peux produire une **étude Phase 1.5 ciblée** sur ce point. - Toujours sans modification de l’architecture tant qu’on n’est pas en Phase 2 validée.

### Sources consolidées

Les sources ci-dessous ont été consultées (ou tentées) pour ce rapport. Celles marquées [WebFetch BLOQUÉ] n’étaient pas accessibles directement mais leurs snippets sont remontés via WebSearch.

**Source canonique** : - [llm-wiki · Gist Karpathy](#) — [WebFetch BLOQUÉ — reconstruit via WebSearch]

**Vidéos YouTube référencées dans le brief** : - [m.youtube.com/watch?v=mVtnN3Jo8ss](#) — [transcript non accessible, contenu reconstruit via recherches sur “Karpathy LLM Wiki walkthrough”] - [m.youtube.com/watch?v=pUIxQ9wj2eA](#) — [transcript non accessible]

**Articles tiers** : - [VentureBeat — Karpathy shares LLM Knowledge Base architecture](#) — [BLOQUÉ] - [MindStudio — What Is Karpathy’s LLM Wiki?](#) — [BLOQUÉ, snippets OK] - [MindStudio — LLM Wiki vs RAG](#) — [BLOQUÉ, snippets OK] - [Antigravity — Complete Guide to Karpathy’s Idea File](#) — [BLOQUÉ] - [PyShine — Claude + Obsidian Self-Organizing Knowledge Engine](#) — [BLOQUÉ] - [Nandigam Substack — Full Breakdown](#) — [BLOQUÉ] - [Medium Urvil Joshi — Create your own knowledge base](#) — [BLOQUÉ] - [Aaron Fulkerson — LLM Wiki in Production \(12/04/2026\)](#) — [snippets OK] - [Scaling LLM Knowledge Bases — Zafer Dace retour terrain \(17/04/2026\)](#) — [snippets OK] - [Starmorph — Complete Guide](#) — [snippets OK] - [AI Maker Substack — AI-Powered Second Brain](#) — [snippets OK] - [Level Up Coding — Beyond RAG](#) — [snippets OK]

**Implémentations open-source de référence** : - [AgriciDaniel/claude-obsidian](#) — [BLOQUÉ, snippets OK] - [lucasastorian/llmwiki](#) — Open-source, MCP - [kytmanov/obsidian-llm-wiki-local](#) — 100% local

avec Ollama - [Pratiyush/llm-wiki](#) — Multi-CLI (Claude, Codex, Copilot, Cursor, Gemini) - [flsteven87/llm-wiki-mcp](#) — Serveur MCP dédié - [ScrapingArt/Karpathy-LLM-Wiki-Stack](#) — Stack de référence - [Astro-Han/karpathy-llm-wiki](#) — Agent Skills compatible - [skyllwt/OmegaWiki](#) — Architecture complète full-lifecycle

**Home Assistant Community — directement pertinent :** - [Filesystem MCP Server](#) — expose your local directory to Claude (Karpathy LLM wiki for Home Assistant) — add-on HA dédié - [Claude Code for Home Assistant](#) - [MCP Server for HA + Local LLM Safety Officer](#)

**Critiques documentées :** - [Dev.to](#) — Hjarni “LLM Wiki is right but I didn’t want to run it locally” - [Dev.to](#) — KIOKU v0.4.0 three things wrong

---

*Fin du Rapport Phase 1 — Étude uniquement — Aucune modification d’architecture Jarvis effectuée.*